

Guia Definitivo de Arquitetura e Deploy: Dashboard Tático

Como desenvolvedor sênior, a primeira coisa que fazemos ao finalizar um MVP (Produto Mínimo Viável) é organizar a "casa". Um MVP foca em provar que a ideia funciona, mas para que o projeto possa crescer, receber novas funcionalidades (como gráficos de desempenho ou integração com APIs) e não virar uma bagunça incontrolável (o famoso *código espaguete*), precisamos de uma base sólida.

Abaixo está a arquitetura ideal e escalável para o seu projeto. Estamos separando rigorosamente as responsabilidades: **Estrutura (HTML)**, **Estilo (CSS)** e **Lógica/Comportamento (JavaScript modular)**. Esta separação não apenas facilita a leitura, mas permite que você isole bugs mais rapidamente e, no futuro, trabalhe em equipe sem gerar conflitos de código.

1. Sistema de Arquivos (Árvore de Diretórios)

A organização das pastas reflete a arquitetura do software. Crie esta estrutura no seu ambiente de desenvolvimento local:

```
projeto-creci/
├── index.html           # Arquivo principal (ponto de entrada da aplicação)
├── assets/             # Diretório de recursos estáticos
│   ├── css/           # Todo o estilo da aplicação
│   │   └── style.css
│   ├── img/           # (Novo) Imagens, ícones locais e logotipos
│   │   ├── favicon.ico # Ícone da aba do navegador
│   │   └── logo-creci.png # Exemplo de imagem
│   └── js/            # Arquivo principal de JS (Orquestrador/Maestro)
│       ├── main.js
│       └── modules/   # Lógica separada por responsabilidade e domínio
│           ├── state.js # Gerencia os dados e o estado global da aplicação
│           ├── storage.js # Lida com LocalStorage, persistência e arquivos
│           ├── ui.js    # Funções exclusivas para manipular a interface (DOM)
│           └── utils.js  # Funções puras e auxiliares (cálculos matemáticos,
```

Por que essa estrutura?

- **Isolamento de Domínio:** Cada arquivo tem um propósito único e bem definido (Single Responsibility Principle).
- **Escalabilidade:** Se o seu CSS crescer muito, você pode criar uma pasta `css/components/` e dividir os estilos. A base de pastas `assets` suporta esse crescimento natural.

2. Refatoração do Código e Padrões de Projeto (Módulos ES6)

O uso de módulos nativos do JavaScript (ES6 Modules) revolucionou o front-end. Ao utilizar `import` e `export`, nós criamos escopos isolados. Isso significa que uma variável chamada `data` no arquivo `state.js` não vai sobrescrever acidentalmente uma variável `data` no arquivo `ui.js`.

Nota de Desenvolvimento Local: Para que os módulos funcionem na sua máquina, o navegador exige que a página seja servida via protocolo `http://` (por questões de segurança do CORS), e não `file://`. **Você precisará usar uma extensão como o "Live Server" no VS Code** (clique com o botão direito no `index.html` e selecione "Open with Live Server").

`index.html` (A Casca)

O HTML torna-se extremamente limpo e semântico. Removemos as tags `<style>` e `<script>` gigantes. Repare na inclusão do atributo `type="module"`, que avisa ao navegador para habilitar o sistema de importações.

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dashboard Tático - CRECI BA | PST</title>

  <!-- CSS Externo -->
  <link href="[https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css]" rel="stylesheet">
  <link href="./assets/css/style.css" rel="stylesheet">
</head>
<body>
  <!-- (TODO O CONTEÚDO HTML FICA AQUI IGUAL AO ORIGINAL) -->

  <!-- JS Externo usando Módulos -->
  <script type="module" src="./assets/js/main.js"></script>
</body>
</html>
```

`assets/css/style.css` (A Pintura)

Recorte todo o conteúdo que estava dentro da tag `<style>` original e cole aqui. Ao centralizar o CSS, o navegador consegue fazer o cache deste arquivo. Se o usuário recarregar a página, o carregamento será instantâneo.

`assets/js/modules/state.js` (Single Source of Truth)

Este módulo é a nossa "Única Fonte de Verdade". Ele guarda as informações e a configuração básica. Qualquer parte do app que precise saber quais são as abas ou os checklists, deve consultar este arquivo.

```
// Exportamos os dados para que outros arquivos possam consumi-los
export const state = {
  activeTab: 'fases',
  checklistData: [
    { id: 't1', category: 'Base', text: 'Tópico de Estudo 1' },
    { id: 't2', category: 'Base', text: 'Tópico de Estudo 2' },
    { id: 't3', category: 'Específicas', text: 'Tópico de Estudo 3' }
  ]
};

// Configurações globais (constantes em MAIÚSCULO como boa prática)
export const CONFIG = {
  examDate: '2026-06-27T00:00:00',
  appVersion: '1.0.0'
};
```

assets/js/modules/utils.js **(Funções Puras)**

Aqui colocamos "Funções Puras", ou seja, funções que apenas recebem dados (parâmetros), fazem um cálculo e devolvem um resultado. Elas não alteram a tela e não salvam nada, o que as torna fáceis de testar.

```
// Retorna a data de hoje no formato YYYY-MM-DD
export const getTodayStr = () => {
  const d = new Date();
  return `${d.getFullYear()}-${String(d.getMonth()+1).padStart(2, '0')}-${String(d.getDate()+1).padStart(2, '0')}`;
};

// Calcula a diferença de dias entre duas datas string
export const getDaysDiff = (dateStr1, dateStr2) => {
  const d1 = new Date(dateStr1 + 'T00:00:00');
  const d2 = new Date(dateStr2 + 'T00:00:00');
  return Math.floor((d2 - d1) / 86400000); // 86400000 ms em um dia
};
```

assets/js/modules/storage.js **(O Wrapper Pattern)**

Tudo relacionado a salvar, recuperar, exportar e importar dados (Backup). Usar este arquivo como um "Wrapper" (Embrulho) ao redor do `localStorage` nativo é uma prática sênior. Se amanhã você decidir trocar o `localStorage` por um banco de dados em nuvem (como Firebase), você só precisará alterar o código dentro deste arquivo, e o resto da aplicação continuará funcionando sem saber da mudança.

```
import { state } from './state.js';

export const storage = {
  // Métodos wrapper de proteção
  saveItem: (key, value) => localStorage.setItem(key, value),
  getItem: (key) => localStorage.getItem(key),
  removeItem: (key) => localStorage.removeItem(key),
};
```

```
// Lógica complexa de exportação de JSON isolada aqui
exportData: () => {
  const dataToExport = {};

  // Salva estado dinâmico do checklist
  state.checklistData.forEach(item => {
    const val = storage.getItem(item.id);
    if (val !== null) dataToExport[item.id] = val;
  });

  // Salva métricas de usuário e gamificação (streaks/fases)
  const metrics = ['lastStudyDate', 'currentStreak', 'bestStreak', 'phase-1'];
  metrics.forEach(key => {
    const val = storage.getItem(key);
    if (val !== null) dataToExport[key] = val;
  });

  // Processo de download simulado via link <a>
  const dataStr = "data:text/json;charset=utf-8," + encodeURIComponent(JSON.stringify(dataToExport));
  const a = document.createElement('a');
  a.href = dataStr;
  a.download = `backup_estudos_${new Date().toISOString().split('T')[0]}.json`;
  document.body.appendChild(a);
  a.click();
  a.remove();
}
};
```

assets/js/modules/ui.js **(A Camada de Apresentação)**

Este módulo é o único autorizado a tocar no Document Object Model (DOM). Essa regra previne bugs difíceis de rastrear. Se algo quebrou na tela, você sabe exatamente em qual arquivo procurar.

```
import { state } from './state.js';

// Função fábrica para obter as referências atuais da tela
export const getDOM = () => ({
  tabButtons: document.querySelectorAll('.tab-btn'),
  tabContents: document.querySelectorAll('.tab-content'),
  checklistContainer: document.getElementById('checklist-container'),
  // ... adicione as outras referências aqui
});

export const ui = {
  switchTab: (targetId, DOM) => {
    state.activeTab = targetId;
    // Alterna classes CSS visualmente
    DOM.tabButtons.forEach(btn => btn.classList.toggle('active', btn.dataset.tab === targetId));
    DOM.tabContents.forEach(content => content.classList.toggle('active', content.dataset.tab === targetId));
  },

  renderChecklist: (DOM) => {
    DOM.checklistContainer.innerHTML = '';
    // (Copiar lógica de renderização do checklist com a reordenação aqui)
  }
};
```

```
    }
  };
}
```

assets/js/main.js (O Orquestrador)

O Maestro. Ele não faz o trabalho pesado sozinho, mas diz a quem deve fazer e quando. Ele junta o estado, a interface, o armazenamento e amarra os eventos de clique do usuário.

```
import { state, CONFIG } from './modules/state.js';
import { getDOM, ui } from './modules/ui.js';
import { storage } from './modules/storage.js';

// Função de inicialização
const init = () => {
  const DOM = getDOM();

  // Setup inicial de ordenação
  state.checklistData.forEach((item, index) => item.originalIndex = index);

  // Primeira renderização da interface
  ui.renderChecklist(DOM);

  // Bind (Atribuição) de Eventos - Escutadores de clique
  DOM.tabButtons.forEach(btn => {
    btn.addEventListener('click', e => ui.switchTab(e.currentTarget.dataset.t));
  });

  // ... bind de outros eventos (export, import, checklist)

  console.log("🚀 Dashboard Tático inicializado com sucesso! Rumo à aprovação!");
};

// Gatilho: Só começa o código quando o HTML inteiro estiver pronto
document.addEventListener('DOMContentLoaded', init);
```

3. Hospedagem Gratuita e Contínua (Deploy na Nuvem com CI/CD)

Colocar uma aplicação estática (HTML/CSS/JS) na internet hoje em dia é fácil, seguro e gratuito. Como devs sêniores, não enviamos arquivos por FTP. Nós usamos esteiras de **CI/CD (Continuous Integration / Continuous Deployment)**. Isso significa que sempre que você salvar e enviar uma mudança para o GitHub, o site atualizará automaticamente na internet.

As melhores plataformas do mercado para isso são **Vercel** e **Netlify**. Recomendo fortemente a **Vercel** devido à sua CDN (Rede de Distribuição de Conteúdo) global, que faz o site abrir muito rápido em qualquer lugar do mundo.

Fluxo Profissional via Vercel (Recomendado):

1. Gestão de Código no GitHub:

- Crie uma conta no [GitHub.com](https://github.com).

- Baixe e instale o Git na sua máquina, ou apenas use o GitHub Desktop / interface do site para criar um repositório novo chamado `dashboard-creci` .
- Faça o upload ou commit da sua pasta local (exatamente a estrutura que criamos) para este repositório.

2. Automação na Vercel:

- Acesse [Vercel.com](https://vercel.com) e crie uma conta (escolha a opção *Continue with GitHub*).
- No painel, clique no botão preto **"Add New"** e depois em **"Project"**.
- A Vercel solicitará acesso à sua conta do GitHub e mostrará seus repositórios. Encontre o `dashboard-creci` e clique em **"Import"**.
- O painel de configurações de Build aparecerá. Como você está usando HTML puro e Módulos Nativos (sem React ou Angular no momento), **não precisa mudar nada**. Deixe o *Framework Preset* como "Other".
- Clique em **"Deploy"**.
- **Mágica feita!** Em menos de 1 minuto, a Vercel compila os dados, cria os certificados de segurança (HTTPS) e te entrega um link público como `dashboard-creci.vercel.app` .
- *Bônus:* Qualquer alteração futura que você enviar para a ramificação `main` do GitHub será atualizada no ar automaticamente em segundos.

Alternativa Rápida (Netlify Drop):

Se você quiser apenas ver o site no ar instantaneamente sem configurar repositórios Git, o Netlify tem uma solução incrível:

1. Acesse a ferramenta [Netlify Drop](https://netlify.com).
2. Pegue a pasta `projeto-creci` do seu computador e arraste para dentro do círculo tracejado no navegador.
3. Em segundos, seu site recebe um link de produção provisório. Criando uma conta logo depois, você consegue mudar o link para algo como `meu-dashboard-creci.netlify.app` .

4. Checklist de Qualidade QA (Quality Assurance Sênior)

Antes de considerar o projeto finalizado e divulgar o link, repasse este checklist rigoroso:

- 

Favicon e Identidade Visual: O site tem um ícone próprio na aba do navegador? Não deixe o ícone padrão do globo terrestre. Adicione `<link rel="icon" type="image/png" href="/assets/img/favicon.png">` no `<head>` .

- 

Meta Tags SEO e Open Graph: Quando você enviar o link no WhatsApp, ele precisa gerar um card bonito (com imagem e descrição). Inclua meta tags de Open Graph (`og:title` , `og:description` , `og:image`) no seu HTML.

- *x*

Acessibilidade (a11y): Usuários de leitores de tela conseguem navegar? Verifique se todos os botões possuem textos claros ou a propriedade `aria-label` (especialmente se o botão for apenas um ícone, como `<button aria-label="Limpar Checklist"><i class="fas fa-undo"></i></button>`). Certifique-se de que o contraste de cores das fases esteja adequado.

- *x*

Responsividade Total (Mobile-First): Teste rigorosamente redimensionando a janela do navegador. Abra as ferramentas de desenvolvedor (F12) e simule um iPhone SE e um iPad. Não deve haver barras de rolagem horizontais.

- *x*

Performance (Lighthouse): Abra o Google Chrome, aperte F12, vá na aba "Lighthouse" e rode um relatório. A meta para um projeto estático como esse é tirar nota verde (90+) em tudo: Performance, Acessibilidade, Boas Práticas e SEO.

- *x*

Progressive Web App (PWA) Básico: Já que o aplicativo salva dados localmente, ele é o candidato perfeito para virar um App Instalável. Adicione um arquivo `manifest.json` na raiz configurando o nome e as cores, e o navegador oferecerá ao usuário de Android/iOS o